

Solaris Submission Q and A

Concise form-ready answers. Each answer is under 4000 characters.

Project: Solaris	Domain: Energy / Solar customer operations	Type: Agentic AI assistant
-------------------------	---	-----------------------------------

1. What's the problem this agent is trying to solve?

Solaris solves a real operational problem in the solar industry: solar users and solar companies have data, but they often do not have timely, intelligent action. A customer may have generation data, appliance usage, bills, cleaning needs, service issues, WhatsApp alerts, and maintenance tickets spread across different systems or manual workflows. Most dashboards only show what happened. They do not explain why production dropped, when to use appliances, whether cleaning is worth it, or when a service ticket should be raised. Solaris closes the gap between solar monitoring and solar action by observing site data, reasoning over it, recommending actions, requesting approval when needed, executing workflows, and tracking outcomes.

2. What specific friction exists that a standard linear program or a human can't solve efficiently?

The friction is that solar customer operations are dynamic, cross-functional, and context-dependent. A fixed rule can flag low production, but it cannot easily reason across weather, appliance load, expected generation, cleaning history, customer preferences, notification opt-ins, and service ticket status. Humans can do this, but not efficiently across hundreds or thousands of customers. Solaris reduces this friction by coordinating specialist agents and tools to observe data, reason over context, select the right workflow, ask for approval, execute actions, verify completion, and remember the outcome.

3. Solution Architect - Provide a high-level overview of how the agent/agentic workflow thinks and acts. Agent Persona: Define the role.

Solaris is an Agentic AI Solar Customer Operations Assistant. Its role is to act like a proactive solar operations co-pilot for an existing solar user and, in the future, for solar businesses managing many customer sites. It follows an observe, reason, select agents, approval gate, act, verify, and learn workflow. Solaris reads solar generation, consumption, appliance data, cleaner status, bill data, weather signals, and open tickets. It compares actual generation against expected production, evaluates appliance load patterns, estimates cleaning ROI, identifies underperformance risk, and decides whether the issue needs a recommendation, approval, or escalation. It then triggers safe tools such as WhatsApp, email, calendar reminders, cleaning schedules, appliance updates, and ticket creation.

4. Solution Architect - The Toolset: List the specific APIs or functions the agent can trigger.

Solaris can trigger customer-safe tools and backend APIs such as GET /api/state, POST /api/agent/chat, POST /api/suggestions/refresh, POST /api/preferences, POST /api/appliances, DELETE /api/appliances/:id, POST /api/cleaner/command, POST /api/cleaner/schedule, POST /api/tickets, POST /api/email/report, POST /api/whatsapp/report, POST /api/whatsapp/test, POST /api/whatsapp/inbound, and POST /api/calendar/plan. When OpenAI is configured, Solaris can safely call tool functions including get_solaris_state, add_appliance, create_ticket, schedule_cleaning, suspend_cleaning, send_email_report, send_whatsapp_alert, and create_calendar_plan.

5. Solution Architect - The Technical Stack

Solaris is built as a web-based Agentic AI prototype using an HTML, CSS, and JavaScript single-page dashboard with a Node.js backend API. The frontend includes overview graphs, appliance tracking, panel cleaning, maintenance tickets, AI Agents, Mission Control, notification settings, and a floating Solaris chat agent. The backend manages local REST-style APIs, customer sessions, state separation, event logs, preferences, tickets, and agent memory. The agentic layer includes a Solaris Orchestrator Agent and specialist agents such as Solar Bill Explainer, Solar Surplus Automation, Cleaning ROI Agent, Underperformance Diagnosis, Peer Benchmark Agent, Notification Agent, Ticket Agent, and Cleaning Agent. Optional integrations include OpenAI Responses API, Gmail SMTP, Twilio WhatsApp, and calendar reminder generation.

6. Business Impact - What value does this bring?

Solaris turns solar monitoring into intelligent solar action. For customers, it improves the ownership experience by explaining performance, appliance usage, billing changes, cleaning needs, EV charging windows, and service issues. For solar businesses, it reduces repetitive support work by preparing tickets with evidence, requesting approvals, sending WhatsApp or email updates, and tracking follow-up. The value is faster issue detection, lower manual service effort, better solar self-consumption, improved customer trust, and a scalable customer operations workflow. Solaris is not just a chatbot; it is an Agentic AI orchestration layer for solar customer experience, service efficiency, and distributed energy operations.

7. Business Impact - What are some quantitative and qualitative ROI insights?

Quantitative ROI includes faster issue detection, potentially reducing detection time from days to minutes or hours; 20-40% less manual follow-up through automated alerts, ticket preparation, approvals, and reminders; 5-15% higher solar self-consumption by shifting EV charging and flexible appliances into solar hours; lower avoidable grid import; and better service turnaround through evidence-ready tickets. Qualitative ROI includes higher customer confidence, clearer decision-making, proactive communication, consistent service quality, reduced operational friction, and stronger enterprise readiness as the same workflow scales across many customers, sites, teams, and integrations.

8. Future steps - How could this scale or grow?

Solaris can scale from a customer-facing prototype into a full Agentic AI platform for solar customer operations. The next steps are real integrations with inverter APIs, smart meters, smart plugs, batteries, EV chargers, weather APIs, CRM systems, ticketing platforms, and cleaner hardware. It can add persistent storage, multi-tenant architecture, background jobs for monitoring and SLA follow-up, role-based access for customers and service teams, and additional specialist agents such as Battery Optimization, EV Charging, Warranty Renewal, Tariff Optimization, Installer Performance, and Customer Retention. Solaris can grow from one solar user to many customers, solar company operations, and an enterprise energy platform.

9. Future steps - Who will use this solution?

The primary users are residential and small commercial solar customers who want to understand generation, track appliance consumption, receive recommendations, approve service tickets, schedule cleaning, and get WhatsApp or email reports. The second user group is solar installation and service companies that need to monitor customer sites, detect underproduction, prepare service tickets, follow up on open issues, and reduce manual support workload. Future users include operations and maintenance teams, installers, warranty teams, call center agents, property managers, housing societies, energy retailers, aggregators, and distributed energy companies.

10. Access Instructions - If sharing a live prototype or agent, mention access clearly.

Solaris is available as a local live prototype. To run it, open PowerShell, go to C:\Users\warri\OneDrive\Documents\Solaris, and run npm start. Then open <http://localhost:8787> in a browser. Demo login: Email customer@solaris.local and Password Solaris@123. The dashboard includes overview charts, appliance tracking, panel cleaning controls, maintenance tickets, AI Agents, Mission Control, orchestration path, timeline, agentic replay, and the floating Solaris Agent chat. Optional integrations include OpenAI API, Gmail SMTP, Twilio WhatsApp, and calendar reminder links; if they are not configured, Solaris still runs with demo and preview behavior.